Ders 4 Genişletilmiş Notu: Görüntü Sıkıştırma: RLE, Faks Standartları, JPEG-LS, JPEG, JPEG 2000 ve Görüntü Formatları

Ders 4 Genişletilmiş Notu: Görüntü Sıkıştırma: RLE, Faks Standartları, JPEG-LS, JPEG, JPEG 2000 ve Görüntü Formatları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Görüntü Sıkıştırmaya Giriş
- 2. RLE ve Görüntü
- 3. Faks Standartları: Modified Huffman ve MMR
- 4. ZIP, GZIP ve PK Programları
- 5. JPEG-LS: Kayıpsız JPEG
- 6. JPEG: Genel Özellikler
- 7. JPEG Mimarisi
- 8. Renk Dönüşümü ve Örneklemeyi Azaltma
- 9. DCT
- 10. Nicemleme, Kodlama ve Ters İşlem
- 11. JPEG 2000
- 12. GIF, TIFF ve PNG
- 13. Kapanış: Görüntü Sıkıştırmanın Arka Planı
- Örnek Sorular
- Dersin Toparlaması

1. Görüntü Sıkıştırmaya Giriş

Hoca derse, görüntü sıkıştırmanın kısa bir konu olduğunu söyleyerek başladı. Bunun nedeni, görüntüde kullanılan yöntemlerin metinde kullanılan yöntemlerin “aynası” olmasıdır. Tek fark, buradaki verinin pikseller ve renkler olması ve kayıplı sıkıştırma yapılacaksa öncesinde bazı ön işlem adımlarının bulunmasıdır.

1.1 Kayıplı ve kayıpsız görüntü sıkıştırma

Metinde olduğu gibi görüntüde de kayıplı ve kayıpsız yöntemler vardır. Slayttaki tanımlar şöyleydi (ekrandan, 00:07:00):

- **Kayıpsız:** Genellikle pikseller arası korelasyonun düşük olduğu şekil gibi görüntülerde kullanılır. Hoca bunu renkler arasındaki farkların fazla olmadığı resimler diye açıkladı.
- **Kayıplı:** Fotoğraf görüntüleri gibi karmaşık ve yüksek korelasyona sahip görüntülerde kullanılır.

Kayıplı sıkıştırmanın gerekçesi gözün sınırlılığıdır. True color denen 24 bitlik bir fotoğrafta 2^{24} (yaklaşık 16,7 milyon) farklı renk olabilir. Göz bu kadar farklı rengi algılayamadığı için bu tür resimlerde kayıplı sıkıştırma kullanılabilir.

1.2 Kayıpsız sıkıştırmada kullanılan yöntemler

Slayta göre kayıpsız görüntü sıkıştırma olasılık tabanlı ya da sözlük tabanlı bir yöntemle yapılabileceği gibi, ikisini bir arada kullanan hibrit bir yöntemle de yapılabilir (ekrandan, 00:07:00). Olasılık tabanlı yöntemler aritmetik kodlama ve Huffman kodlaması, sözlük tabanlılar LZ77 ve LZW gibi LZ türevleridir.

Hoca örnek olarak PNG'yi verdi. PNG ücretsiz, lisans hakkı olmayan ve iyi sıkıştırma yapan bir resim formatıdır. İçinde LZ77 ve Huffman birlikte kullanılır; bu hibrit kullanıma DEFLATE sıkıştırma yöntemi denir. Hoca burada LZ77'yi kısaca hatırlattı: Kayan pencerenin bir tarafından veri girer; pencere içindeki daha önce işlenmiş veri sözlük olarak kullanılır. Yeni gelen veri pencerede en uzakta nerede bulunuyorsa onun offset değeri, eşleşmenin uzunluğu ve ardından gelen sembol (next symbol) yazılarak token oluşturulur.

Slaytta ayrıca JPEG-LS ve JBIG'in özellikle kayıpsız görüntü sıkıştırma için tasarlanmış yöntemler olduğu yazıyordu (ekrandan, 00:07:00). Hoca bunlardan JPEG-LS'yi biraz daha ayrıntılı anlatacağını söyledi.

2. RLE ve Görüntü

2.1 RLE'nin mantığı

Hoca RLE'yi daha önce ödev olarak verdiğini hatırlattı ve RLE'nin amacını sordu. RLE, bir veride bazı semboller art arda geliyorsa ve bu art arda gelişler çok sık tekrar ediyorsa iyi performans verir. Slayttaki tanım (ekrandan, 00:11:45): "RLE'nin temel mantığı, bir değerin ardışık olarak çok sayıda tekrar etmesi durumunda, o değeri bir defa kodlayıp ardından tekrar sayısını kodlamaktır."

Metinde RLE başarısızdır, çünkü bir metinde aynı karakterin art arda ikiden fazla tekrar etmesi pek mümkün değildir. Tekrar edebilecek olanlar boşluk karakteri ya da çok sıfırlı bir sayıdır. Hoca bunu bir örnekle hatırlattı: Art arda 50 boşluk varsa koşu (run) değeri olarak 50, ardından boşluk karakteri yazılır; çözücü bundan 50 boşluk olduğunu anlar.

Slayta göre RLE her tür veri için kullanılabilse de genellikle görüntü sıkıştırmada kullanılır; BMP, PCX ve TIFF görüntü dosya formatları RLE ile sıkıştırma yapar (ekrandan, 00:11:45). Hoca bunun nedenini bu formatların desteklediği renk sayısının daha kısıtlı olmasına bağladı: JPEG 2^{24} farklı rengi desteklerken bu formatlarda renk sayısı daha azdır, aynı renkteki piksellerin art arda gelme sıklığı daha yüksektir ve RLE iyi sonuç verir.

> **Ders sonrası not:** Hoca BMP ve TIFF'in o kadar çok rengi desteklemediğini söyledi (00:12:38). Dersin sonundaki TIFF slaytına göre TIFF, 24 bit RGB renk derinliğini de destekler (ekrandan, 01:16:30).

2.2 Kayıplı RLE

RLE kayıplı olarak da kullanılabilir. Bir rengin koşu değeri iki ya da üç gibi küçükse, yani bir renk yalnız iki-üç kez art arda geçiyorsa, bu kısa koşu ihmal edilir ve bu pikseller komşusu olan en uzun koşunun rengiyle kodlanır. Bunun dayanağı yine gözdür: İki farklı kırmızı tonu yan yana geldiğinde göz bunları aynı görür. Yirmi farklı kırmızı piksel yan yana geldiğinde bile ayırt etmekte zorlanılıyorsa, az sayıdaki pikselin renkleri uzun koşunun rengine çevrilebilir.

2.3 Siyah-beyaz satır örneği

Slayttaki örnek (ekrandan, 00:11:45): 1 bitlik renk derinliğine sahip bir görüntü yalnız iki renk içerir: siyah ve beyaz. Bir satırdaki 640 pikselin renkleri sırasıyla 200 beyaz, 240 siyah ve 200 beyaz ise satır RLE ile $(200, 240, 200)$ şeklinde kodlanır.

Burada renkler yazılmaz, yalnız tekrar sayıları yazılır. Bunun çalışması için çözücünün ilk pikselin rengini bilmesi gerekir. Slayta göre ilk pikselin rengi kodlamanın başında belirtilir ya da ilk piksel her zaman beyaz kabul edilir. Siyah-beyaz resimlerde varsayılan kural, ilk okunan değerin beyaz olmasıdır; ardından değerler beyaz, siyah, beyaz diye sırayla gider.

Resim beyazla başlamıyorsa başa 0 yazılır. Hoca bunu şöyle anlattı: “Sıfır tane beyaz var, iki yüz tane siyah, iki yüz kırk tane beyaz, iki yüz tane siyah var.” Gri tonlu bir resim söz konusuysa ve resim beyazla başlamıyorsa, hangi renkle başladığı bilgisi başlıkta (header) bir yere yerleştirilir. Çözücü bu bilgiyi başlığın hangi byte’ından okuyacağını bildiği için oradan okuyup karar verir.

3. Faks Standartları: Modified Huffman ve MMR

3.1 Faks neden görüntüdür?

Hoca daha önce metin sıkıştırma içinde anlatılan faks yöntemlerini hatırlattı. Faks makinesine konan bilgi bir metindir; ama faks makinesi, içeriği metin olsa bile sayfayı bir resim dosyası olarak görür. Siyah yazılar resimdeki siyah piksellere, beyazlar beyaz piksellere karşılık gelir.

Faksta piksel yerine pel denir. Hocanın açıklamasına göre bunun nedeni, bir satıra düşen pel sayısının fazla olması ve satırın bir karakteri yazmak için gereken yüksekliğe göre tanımlanmasıdır. Bir karakterin bir eni ve boyu vardır; bunlar pel’lerden oluşur. Bir karakterin boyuna kaç pel geliyorsa satırın yüksekliği odur.

3.2 Modified Huffman (G31D)

Slayttaki bilgiler (ekrandan, 00:20:00):

- Fax makinelerinde kullanılan CCITT T4 (Group 3) standardı RLE ve Huffman kodlaması kullanır.
- İlk piksel her zaman beyaz kabul edilir.
- Tekrar sayıları çok farklı olasılıklara sahip olduğu için 64’ten büyük tekrar sayıları önce 64’e bölünür; bölüm ve kalan farklı Huffman tablolarıyla kodlanır.
- Bu kodlama Modified Huffman (MH) veya Group 3 One-Dimensional (G31D) olarak bilinir.

Hoca iki tabloyu hatırlattı: Birinde 0’dan 63’e kadar 64 değeri alan kodlar, diğerinde 64’ün katlarına verilmiş Huffman kodları vardır. Bir koşu uzunluğu 64’e göre mod alınarak ayrılır: kalan birinci tablodan, 64’ün katı ikinci tablodan kodlanır. Tablonun bu iki parçalı yapısı yüzünden yönteme “Modified Huffman” denir.

> **Ders sonrası not:** Hoca örneği “55 tane koşu değeri gelmişse 54’e göre mod alıyordum, 1 kalıyordu; 64’ün tek katı olduğu için 64’ün katına bakıyordum” diye söyledi (00:18:02). Anlatılan tablo yapısına göre örnek büyük olasılıkla 65 uzunluğundaki bir koşudur: $65 = 64 \cdot 1 + 1$, yani 64 için ikinci tablodan, 1 için birinci tablodan kod alınır.

3.3 Faksta neden kayıpsız?

Faks bir metin gönderdiği için sıkıştırma kayıpsız olmak zorundadır. Kayıplı sıkıştırmada karakterlerde kopukluklar olur. Hoca bunu ekranda bir “T” harfi çizerek gösterdi (ekrandan, 00:20:00): Kayıp, T’nin üst çizgisinin bir kısmına denk gelirse karşı tarafta “L” gibi bir şekil kalır ve harfin T mi L mi olduğu anlaşılamaz. Bunu düzeltmek için doğal dil işleme uygulamak, yani n-gram’lara bakarak oradaki karakterin ne olması gerektiğini tahmin etmek gerekir. Bu yüzden faksta kayıpsız sıkıştırma kullanılır.

3.4 MMR (G32D)

Slayttaki bilgiler (ekrandan, 00:20:00): CCITT T4 standardı, IBM tarafından 1979 yılında geliştirilen MMR (Modified Modified READ) sıkıştırmasını da kullanabilir. Bu, Group 3 Two-Dimensional (G32D) olarak bilinir. Bir önceki satır dikkate alınarak daha yüksek sıkıştırma oranlarına ulaşılır. MMR’nin sıkıştırma performansı, daha sonra geliştirilen JBIG standardının gerisinde kalmıştır.

Hoca iki boyutlu kodlamayı şöyle hatırlattı: Pel’lerden oluşan iki satır vardır; üstteki referans alınan satır, alttaki kodlanan satırdır. Pointer’ların (a1, b1 gibi) konumlarına bakılır; örneğin

a1'in b1'in solunda ya da sağında olmasına göre farklı kiplere girilir. Hoca yatay kip (horizontal mode) ve dikey kipi (vertical mode) saydı, üçüncü kipin adını o an hatırlamadı. Bunların ayrıntısının ezberlenmesi gerekmediğini söyledi.

> **Ek bilgi (derste anlatılmadı):** İki boyutlu faks kodlamasındaki üçüncü kip büyük olasılıkla geçiş kipidir (pass mode). Bu kip, referans satırdaki bir siyah ya da beyaz parçanın kodlanan satırda karşılığı olmadığında kullanılır.

Hoca ardından JBIG standardından söz etti.

4. ZIP, GZIP ve PK Programları

4.1 ZIP ve GZIP

Slayttaki bilgiler (ekrandan, 00:30:00):

- Her bloğun Huffman ağacı ayrıdır ve her bloğun başında sıkıştırılarak saklanır.
- LZ77'nin bir türevi ile statik Huffman kodlamanın birleşimidir.
- 32 KB'lık kayan bir sözlük ve 258 byte'lık kaynak kod alanı kullanır. String sözlükte yoksa olduğu gibi kodlanır.
- Giriş verisi kodlayıcı tarafından bloklara ayrılır. Blok uzunlukları keyfîdir.

Hoca blok yapısını ekranda bir dosya çizerek anlattı. Dosya bloklara bölünür ve her blok Huffman ile ayrı ayrı sıkıştırılır. Her bloğun başına bir başlık eklenir: bloğun uzunluğu ve Huffman'ın başlıkta olması gereken bilgileri yazılır, altına sıkıştırılmış veri gelir. Sonra ikinci blok için yine bir başlık, altına onun sıkıştırılmış verisi gelir ve bu böyle devam eder. Çözücü başlıktan blok boyutlarını okuyarak açar.

"Neden her bloğun boyutu aynı değil?" sorusuna hoca bir örnekle cevap verdi: Elinizde iki kitap sayfası olsun. Birinin yarısı resim, bir kısmı yazı, bir kısmı boşluktur; diğerinin tamamı yazıdır. Her sayfa aynı bloklarla sıkıştırılmaz, içindeki verinin özelliklerine bakılır. Resim, boşluk ve yazı varsa resimler belki ayrı bir algoritmayla sıkıştırılır. O zaman blokların boyutları da, kullanılan yöntem de farklı olur; bu bilgiler de başlığa yazılır.

Bazı bloklarda Huffman yerine LZ77'nin bir türevi kullanılır. LZ77 türevi demek, LZ77'nin bazı parametrelerinin değiştirilmesi demektir: pencerenin boyu, kaynak kod alanının boyu, token'ın iki alanlı mı üç alanlı mı olacağı gibi. Her parametre değişikliği LZ'nin farklı bir türevini (LZY, LZAP, LZW gibi) ortaya çıkarır. ZIP ve GZIP'te genellikle 32 KB'lık kayan pencere (sözlük alanı) ve 258 byte'lık kaynak kod alanı (look-ahead buffer) kullanılır. Kaynak kod alanına işlenmemiş veri gelir; sözlükte varsa token yazılır (şu konumda, şu uzunlukta eşleşme ve ardından gelen byte), yoksa ASCII olarak olduğu gibi kodlanır.

4.2 PK programları

Slayttaki başlık "PKZip (Phil Katz) UnZip, Arc, Lite" idi ve şu satırlar vardı (ekrandan, 00:30:00):

- PK programları birkaç sıkıştırma algoritmasını içerisinde barındırır.
- PKArc → LZW ve Huffman Coding.
- Dinamik LZW, RLE.
- LZ77 ve Shannon Fano.

Hoca LZW'yi hatırlattı: Sözlük kullanan bir yöntemdir. Sözlük alfabedeki harflerle başlar; ardından harflerin ikili kombinasyonlarına bakılır, sözlükte yoksa eklenir ve en uzun kombinasyonlar oluşana kadar devam edilir. Farklı tipte dosyalar (görüntü, ses gibi) sıkıştırılacaksa her biri en uygun yöntemle kodlanır; PKArc bunları LZW ve Huffman ile hibrit olarak kodlar.

Shannon-Fano'yu da kısaca hatırlattı: Olasılık değerleri büyükten küçüğe sıralanır, olasılıklar yarı yarıya olacak şekilde ikiye bölünür, her parça yine ortadan ikiye bölünür ve tek parça kalana kadar devam edilir.

Hoca, görüntü sıkıştırmanın daha kısa sürmesinin nedeninin bu yöntemlerin daha önce görülmüş olması olduğunu, bu yüzden bunları yeniden açık açık anlatmaya gerek olmadığını söyledi.

5. JPEG-LS: Kayıpsız JPEG

5.1 Genel bilgiler

Slayttaki başlık "JPEG-LS (Lossless JPEG) Joint Photographic Experts Group" idi (ekrandan, 00:31:40):

- ISO/IEC tarafından geliştirilmiştir (International Organisation for Standardisation / International Electrotechnical Commission).
- JPEG-LS, görüntüleri kayıpsız veya az kayıplı sıkıştırabilen bir algoritmadır (1998).
- Standardı Hewlett-Packard Laboratories tarafından geliştirilen LOCO-I (LOw COmplexity LOssless COmpression for Images) algoritması oluşturmaktadır.
- Hem renkli hem de gri tonlamalı görüntülerin sıkıştırılmasında kullanılır.
- Gri tonlamalı görüntülerde ne yapılıyorsa, renkli görüntülerde de kırmızı, yeşil ve mavi renk paletleri için aynı mantık kullanılır.
- Kodlayıcı görüntünün sol üst pikselinden başlayarak sağ alt pikseline kadar satır tarama düzeninde işlem yapar.
- Bir pikselin değeri tahmin edilirken kodlayıcı ve çözücüde o pikselin üstündeki ve solundaki piksellerin değerleri mevcuttur.
- JPEG-LS algoritması bu değerlerden sadece 4 tanesini kullanır: soldaki (W), üst soldaki (NW), üstteki (N) ve üst sağdaki (NE).

JPEG'in farklı versiyonlarında kayıp oranı kullanıcı tarafından belirlenebilir: %20, %30 ya da %60 kayıp kabul edilebilir. Hoca bunu iki örnekle açıkladı. Bir MR görüntüsü JPEG'e çevrilirken en ufak bir hata bile kabul edilemeyeceği için kayıpsız (yüzde sıfır kayıplı) çevrilmesi istenir. Deniz ve gökyüzünden oluşan bir resimde ise mavinin birçok tonu vardır; göz bu tonları zaten ayıramayacağı için %30-40 kayıp kabul edilebilir.

5.2 Gri ton ve renkli görüntü

Hoca siyah-beyaz bir resimde JPEG kullanmanın çok mantıksız olduğunu, siyah-beyaz resim için RLE'den daha iyi sonuç verecek bir yöntem olmadığını söyledi.

Gri tonlu görüntülerde gri seviye sayısı bit sayısına bağlıdır: 3 bitlik gri seviyede $2^3 = 8$, 4 bitte $2^4 = 16$, 5 bitte 32, 6 bitte 64 farklı gri tonu vardır. Renkli (true color) bir resimde kırmızı, yeşil ve mavi olmak üzere üç palet vardır ve her biri 8 bittir; $3 \times 8 = 24$ bit eder. JPEG-LS her paleti sanki ayrı bir gri tonlu resimmiş gibi ayrı ayrı kodlar. Açarken kırmızı, yeşil ve mavi birleştirilerek tek bir rengin kodu elde edilir.

5.3 Tarama düzeni

Resim sıkıştırılırken nereden başlanacağı önemlidir. Sayfanın sol en üst pikselinden başlanır. Satır satır, en sağdaki piksele gelinceye kadar okunabilir; kolon kolon da okunabilir; satırlarda ya da kolonlarda yön değiştirerek (rotate) okunabilir ya da zigzag yapılabilir. Hoca, ödevde çoğu öğrencinin bütün resme zigzag uyguladığını hatırlattı. JPEG ise bütün resme tek bir zigzag uygulamaz; resmi 8×8 ya da 32×32 gibi küçük bloklara böler, her bloğa kendi içinde zigzag uygular ve kodlar, en sonunda birleştirir. Açarken ters işlem yapılarak başlangıç noktasına dönülür.

5.4 Komşu pikseller

Bir pikselin sekiz komşusu vardır: kuzey (N), güney (S), batı (W), doğu (E), kuzeybatı (NW), kuzeydoğu (NE), güneydoğu (SE) ve güneybatı (SW). Pikselin kendisiyle birlikte 9 değer vardır. JPEG-LS bunlardan 4 tanesini kullanır: üzerinde çalışılan pikselin solundaki, üstündeki, sol üstündeki ve sağ üstündeki pikseller. Bu pikseller tarama sırasında zaten işlenmiş olduğu için hem kodlayıcı hem çözücü bunları bilir. Hoca bunu ekranda bir ızgara çizerek gösterdi (ekrandan, 00:40:00).

```
NW | N | NE
---+---+---
W  | x |
```

5.5 Birinci aşama: tahmin

Tahmin iki aşamada gerçekleşir. İlk aşamada her piksel değeri için slayttaki tablo kullanılır (ekrandan, 00:40:00):

$$G = \begin{cases} \min(W, N) & \max(N, W) \leq NW \\ \max(W, N) & \min(N, W) < NW \\ W + N - NW & \text{diğer durumlarda} \end{cases}$$

Hoca tabloyu şöyle okudu: Kuzey ve batıdaki piksel değerlerinden büyük olanı NW değerinden küçük ya da eşitse, W ile N'nin küçüğü alınıp G'ye yazılır; yani piksel güncellenir. Slayttaki açıklamalar:

- İlk iki madde dikey ve yatay kenarları yakalar.
- Örneğin $N > W$ ve $N \leq NW$ ise bu yatay kenarı işaret eder ve tahmin olarak W kullanılır.

Hoca sezgiyi şöyle verdi: Bir yönde (yatayda ya da dikeyde) değişim kuvvetliyse piksel renk olarak o yöndeki komşusuna benzetilmeye çalışılır. Ne yatayda ne dikeyde belirgin bir kenar yoksa üçüncü durum geçerlidir; bu durumda hocanın ifadesiyle diyagonaldeki değerler kullanılır ($W + N - NW$).

> **Ders sonrası not:** Slayttaki ikinci satırın koşulu "$\min(N, W) < NW$" diye yazılmıştı; bu koşul ilk satırın koşuluyla çakışır. LOCO-I'nin medyan kenar tahmincisinde ikinci koşul büyük olasılıkla $NW \leq \min(N, W)$ olmalıdır. Slayttaki "son madde ise yatay kenarları yakalar" cümlesi de büyük olasılıkla bir yazım hatasıdır; üçüncü durum belirgin kenar olmayan bölgeler içindir.

5.6 İkinci aşama: düzeltme

Elde edilen G tahmininden sonra resim üzerinden ikinci bir geçiş yapılır ve tahminin çevresindeki piksel çiftlerinin değer farklarına göre düzeltme yapılır (ekrandan, 00:40:00). Üç piksel çifti kullanılır: (NW, W), (NW, N) ve (N, NE). Slayta göre iki bitişik pikselin değerlerinin farkına göre her biri 9 gruptan birine sınıflandırılır. Bu da toplamda $9^3 = 729$ farklı durum ortaya çıkarır. Her durum, tahmini düzeltmek için kullanılan kendine ait düzeltme değerini ve o durum için daha önceki tahminlerin kaliteleriyle ilgili bilgiyi saklar. Algoritma ikinci geçişten sonra sonuç tahminini üretir ve gerçek değerle arasındaki farkı kodlar.

Hoca bu kadar farklı durumla uğraşmak yerine en önemli noktanın yatay ve dikey geçişlerdeki değişimi veren bu üç piksel çiftinin farklarına bakmak olduğunu söyledi. Algoritmanın daha derin ayrıntıları vardır, ama JPEG'i oturup yazmayacağımız için bunlara girmeye gerek olmadığını belirtti.

> **Ders sonrası not:** Hoca 729'u "ortadaki pikselle birlikte 9 durum vardı; 3 çift varsa 9^3" diye açıkladı (00:41:31). Slayta göre buradaki 9, komşu piksel sayısı değil, her çiftin farkının sınıflandırıldığı grup sayısıdır.

6. JPEG: Genel Özellikler

Hoca ardından JPEG'in sade (varsayılan) sürümünü anlattı. JPEG, Joint Photographic Experts Group'un kısaltmasıdır; resimler üzerinde sıkıştırma algoritması geliştirmek isteyen bu grubun çıkardığı bir formattır. Slayttaki maddeler (ekrandan, 00:43:30):

- DCT (Ayrık Kosinüs Dönüşümü) tabanlı bir standarttır.
- Sıkıştırmadaki kayıp oranı (sıkıştırma miktarı) kalite faktörünün (Q faktörü) değiştirilmesiyle ayarlanabilir.
- 24 bit renk derinliğine sahip fotoğraflarda çok iyi sonuçlar verir (sayısal fotoğrafçılıkta kullanılır).
- JPEG, birçok görüntü sıkıştırma yönteminin bir araya getirilmesiyle oluşturulmuş bir standarttır.
- Sıkıştırma ve açma süreleri tek algoritmaya sahip olan kayıpsız sıkıştırma yöntemlerine göre daha uzundur.

Hoca DCT'nin sinyal işleme ya da görüntü işleme dersi alan herkesin karşısına mutlaka çıkacağını söyledi. Q faktörü aslında bir tablodur (matristir). Resim kaça kaçlık bloklara bölündüyse (8×8, 16×16) Q matrisi de o boyuttadır. Örneğin %40 sıkıştırma için bir Q matrisi, %60 için başka bir matris kullanılır. Bu matrisler birçok denemeyle çıkarılmış, hazır olarak sunulmuştur; kullanıcı bunları hazır alır.

JPEG gri tonlu ya da siyah-beyaz resimler için mantıklı değildir; bunlarda diğer yöntemler çok daha iyi sonuç verir. Fotoğraf makinesinde çekilen resimler indirildiğinde genellikle JPEG uzantılı olur, çünkü JPEG kaliteyi çok bozmadan iyi bir sıkıştırma sağlar. Cihazdaki yazılımın yüzde kaçlık sıkıştırma kullandığını bilemeyiz. Hoca bir olasılık olarak şunu söyledi: Zoom yapılarak yakından ve hassas bir şey çekiliyorsa yazılım belki kayıpsız JPEG kullanıyor, gözün zaten net göremeyeceği uzak noktaları içeren bir resimde ise kayıplı bir format seçiyor olabilir; ama arka planda ne yapıldığını bilemeyiz.

Süre konusunda hoca şunu vurguladı: JPEG birden fazla adımdan oluştuğu için işlem yükü daha fazladır ve süresi biraz daha uzun olabilir. Ama bugünkü donanımlarda bir TIFF ile bir JPEG arasındaki açılma süresi farkı algılanamaz; on yıl önceki bilgisayarlarda aynı resmi farklı formatlarda açınca bazılarının daha yavaş açıldığı söylenebilirdi.

7. JPEG Mimarisi

Hoca mimariyi slayttaki şema üzerinden anlattı (ekrandan, 00:50:00). Şemadaki akış şöyleydi:

```
Kaynak dosya
  -> [RGB -> YUV] (isteğe bağlı, kesikli kutu)
  -> Örneklemeyi azaltma
  -> Bloklara bölme (Y, U, V ayrı ayrı; 8x8)
  -> DCT
  -> Niceleme  <- Niceleme tabloları
  -> ZigZag tarama
       DC -> DPCM
       AC -> RLE
  -> QM kodlaması  <- Kodlama tabloları
  -> JPEG dosyası: Başlık | Tablolar | Veri
```

Slayttaki maddeler:

- Renk bilgisi RGB türünde ifade edilmişse YUV'a çevirme (isteğe bağlı).
- Örneklemeyi azaltma (down-sampling).
- Görüntüyü 8×8 büyüklüğünde bloklara bölme.
- DCT kullanarak piksel bilgisini zaman etki alanından frekans etki alanına çevirme.
- DCT dönüşümü sonuç değerlerine nicelendirme.
- Sonuç katsayılara köşegen tarama düzeninde RLE uygulama ve ardından QM kodlamasını kullanma.

Hoca adımları kısaca şöyle özetledi. Kaynak resim RGB formatındadır. İstenirse YUV gibi başka bir formata çevrilir; şemadaki kutunun kesikli çizilmesi bu adımın isteğe bağlı olduğunu gösterir. Sonra örneklemeyi azaltma yapılır. Örneğin dört pikselden oluşan bir parçada dört pikselin yerine bir pikselin rengi dördüne birden konabilir; bu yapıldığında RLE'de hepsi aynı renk koduna gelmiş olur. Ardından Y, U, V paletlerinin her biri ayrı ayrı küçük bloklara bölünür. Hoca burada ödevle bağlantı kurdu: Bütün resme zigzag uygulamak yerine resim küçük parçalara bölünüp her birine zigzag uygulansaydı sıkıştırma performansı daha yüksek olurdu.

> **Hocanın vurgusu:** Resmi küçük bloklara bölüp her bloğa ayrı ayrı tarama ve kodlama uygulamak, bütün resme tek bir tarama uygulamaktan daha yüksek sıkıştırma performansı verir.

Her palete ayrı ayrı DCT uygulanır; DCT, resimdeki yüksek ve alçak frekansları belirler. Ardından nicemleme gelir: %20, %30, %40 gibi bir sıkıştırma isteniyorsa onun için hazırlanmış nicemleme tabloları kullanılır. Bu adımın da şemada kesikli olmasının nedeni, hiç kayıp istenmiyorsa bu kısmın kullanılmamasıdır. Sonra zigzag tarama yapılır; DC ve AC katsayıları ayrılır ve kendi içlerinde ayrı ayrı sıkıştırılır. Kodlanmış veri başlığın altına yazılır.

Başlıkta hangi Q tablosunun kullanıldığı gibi bilgiler bulunur, çünkü açan taraf tam ters işlemi yapacaktır. Çözücü önce başlığa bakarak sıkıştırma (nicemleme) yapılıp yapılmadığını anlar; yapılmışsa tablolardan geçirip küçük matrisleri oluşturur, sonra DCT'nin tersini yapar ve YUV'dan RGB'ye dönerek orijinal resmi elde eder. Seçilen kayıp oranına bağlı olarak orijinal resme tam dönülemeyebilir.

> **Ders sonrası not:** Hoca bu bölümde "gözüm alçak frekanslardaki renkleri göremediği için onlar kayıp olarak gidecek, yüksek frekanslar ise kullanacağım piksel renk değerlerim olacak" dedi (00:49:43) ve "DC'lerin birçoğu sıfır olarak zaten kaybolacak" diye ekledi (00:50:56). Slayta ve hocanın sonraki anlatımına göre insan gözü yüksek frekansları algılayamaz ve atılan bunlardır; sıfırlanan katsayılar yüksek frekanslı AC katsayılarıdır.

8. Renk Dönüşümü ve Örneklemeyi Azaltma

8.1 Neden YUV?

Slayttaki "Renk Dönüştürme" bölümü (ekrandan, 00:52:30):

- RGB'den YUV'a dönüştürme JPEG sıkıştırması için gerekli değildir, ama bu dönüşüm sayesinde yüksek sıkıştırma oranlarına ulaşılır. R, G ve B bileşenleri arasındaki korelasyonu azalttığı için sıkıştırmanın etkinliği artar.
- YUV renk formatı renkleri parlaklık (luminance) ve renklilik (chrominance) olarak ifade eder.
- Y parlaklığı, U ve V değerlerinin kombinasyonları da renkliliği ifade eder.
- RGB→YUV dönüşümü bir lineer dönüşümdür.

Hoca nedeni şöyle açıkladı: Bir pikselin üç palette de değeri vardır. YUV'a geçirildiğinde renk değerleri birbirine yaklaşır ve renk sayısında bir azalma olur. İnsan gözü parlaklığı daha iyi algılar: Parlak bir renkle mat bir renk rahatlıkla ayırt edilir. Renklilikteki çok hassas değişimleri ise göz görmez. Önemli olan parlaklığı korumaktır.

Hoca bunu ekrana iki piksel yazarak gösterdi (ekrandan, 00:55:10): Bir pikselin R, G, B değerleri 45, 55, 42, diğerininki 45, 55, 41. Bu iki piksel arasındaki tek birimlik farkı insan gözü ayırt edemez. Bu tür değişimler başka bir palete geçirildiğinde daha iyi sıkıştırma yapılabilir.

> **Ders sonrası not:** Hoca sözlü anlatımda farkın green paletinde (42 ile 41) olduğunu söyledi (00:54:04); ekrana yazılan değerlerde fark üçüncü bileşendedir (B: 42 ve 41).

8.2 Dönüşüm formülleri

Slayttaki dönüşüm (ekrandan, 00:52:30):

$$Y = 0{,}299R + 0{,}587G + 0{,}114B$$
$$U = -0{,}169R - 0{,}331G + 0{,}500B$$
$$V = 0{,}500R - 0{,}149G - 0{,}081B$$

Pikselin R değeri ilk katsayıyla, G değeri ikinci, B değeri üçüncü katsayıyla çarpılır; toplam Y'yi verir. U ve V aynı R, G, B değerlerinin farklı katsayılarla çarpılmasıyla bulunur. YUV'dan RGB'ye dönmek için ayrı katsayılar vardır; HSV gibi başka renk paletleri için de ayrı çarpanlar bulunur. Hoca bu katsayıların testlerle çıkarılmış hazır değerler olduğunu, görüntü işleme kitaplarında bulunabileceğini ve kimsenin bunları yeniden bulmaya çalışmadığını söyledi.

> **Ders sonrası not:** Slayttaki V satırında $-0{,}149G$ yazıyordu; bu büyük olasılıkla $-0{,}419G$ olmalıdır. Gri bir piksel ($R = G = B$) için renklilik bileşeni sıfır çıkmalıdır. U'nun katsayılarının toplamı $-0{,}169 - 0{,}331 + 0{,}500 = 0$ iken slayttaki V katsayılarının toplamı $0{,}500 - 0{,}149 - 0{,}081 = 0{,}27$'dir; $-0{,}419$ ile toplam sıfır olur (python ile kontrol edildi).

8.3 Örneklemeyi azaltma (down-sampling)

Slayttaki "Örneklemeyi Azaltma" bölümü (ekrandan, 00:52:30):

- İnsan gözü parlaklığa daha çok, renkliliğe daha az hassastır. U ve V değerlerini bir miktar azaltmak, görüntüyü insan gözü için çok fazla farklılaştırmaz.
- Bu sebeple 2×2 ebatlarında 4 piksellik bir blok için Y, U ve V değerlerinden 4'er adet (4:4:4) kullanmak yerine 4:2:2 veya 4:1:1 kullanılabilir. Bu işleme down-sampling denir.

Hoca bunu ekranda 2×2'lik dört nokta çizerek anlattı (ekrandan, 00:57:30). Y parlaklık olduğu ve göz onu iyi algıladığı için dört değer olduğu gibi alınır. U ve V renklilik olduğu için bazen yalnız iki değer alınır (iki pikselin değeri diğer ikisine de kullanılır); bu 4:2:2'dir. Daha az örnek alınacaksa Y'nin dört değeri alınır, ama U ve V'den yalnız birer değer alınır; bu 4:1:1'dir. Örnek azaltılmak istenmiyorsa Y, U ve V için hep dört değer kullanılır (4:4:4).

> **Ders sonrası not:** Hoca 4:4:4 gösterimindeki sayıları "4 bit" diye anlattı (00:57:16). Bu sayılar bit değil, 4 piksellik blokta her bileşen için tutulan örnek (değer) sayısıdır.

9. DCT

9.1 Bloklar ve katsayılar

Slayttaki DCT bölümü (ekrandan, 01:00:00):

- Görüntü 8×8 bloklara bölünür ve her blok diğerinden bağımsız olarak ayrık kosinüs dönüşümüne (DCT, Discrete Cosine Transform) tabi tutulur.
- Sonuçta ortaya yine 64 katsayı çıkacaktır. Bunlardan ilkine DC (ilk piksel, $D(0,0)$), diğerlerine AC adı verilir.
- İnsan gözü yüksek frekanstaki değerleri algılayamaz.
- Resimde sıkıştırma yapılacaksa yüksek frekanslar çıkarılır.
- Resim 8×8'lik bloklara ayrılır (blok içinde renk değerleri 0-255 arasındadır).
- DCT $(-128, 127)$ arasında çalıştığından bütün renk değerlerinden 128 çıkarılır (T matrisi).
- Her bloğa ayrı ayrı DCT uygulanır.
- Düşük frekanslı değerler matrisin sol üst köşesinde, yüksek frekanslı değerler sağ alt köşede yer alır.

Paletler ayrı ayrı işlenir: Red paletinin 8×8'lik blokları, green paletinin blokları ve blue paletinin blokları ayrı ayrı DCT'den geçer.

9.2 Zigzag şekli ve katsayı türleri

Hoca, sonraki slayttaki 8×8'lik renkli şekil üzerinden katsayı türlerini anlattı (ekrandan, 01:01:00). Şekilde hücreler zigzag tarama sırasına göre 0'dan 63'e numaralanmıştır; renklerin anlamı şöyledir:

- Gri (0 numaralı hücre, sol üst): DC katsayısı.
- Koyu mavi (sol üst bölge): değeri 0'dan farklı AC katsayıları.
- Açık gri (sağ alt bölge): değeri 0 olan AC katsayıları.

Köşegenin altında kalan, sağ alt taraftaki bölgedeki katsayılar gözün algılayamadığı, varyansı çok düşük AC katsayılarıdır ve yok edilmeye çalışılır. Zigzag tarama, sol üstteki DC'den başlayıp köşegenler boyunca ilerleyerek sağ alttaki 63 numaralı hücrede biter.

> **Ders sonrası not:** Hoca DCT uygulandığında sol üstteki katsayının "kesinlikle 0 olup DC'yi anlatacağını" söyledi (01:01:32). Şekildeki 0, DC katsayısının zigzag sıra numarasıdır; DC katsayısının değeri sıfır değildir.

9.3 Frekanslar ve göz

İnsan gözü yüksek frekanstaki değerleri algılayamaz; DC düşük frekanstır ve göz tarafından algılanır. Resimde sıkıştırma yapılacaksa göz algılamadığı için yüksek frekanslar resimden

çıkarılır. Matrisin sol üst köşesinde düşük frekanslar, aşağıya ve sağa doğru yüksek frekanslar yer alır. Köşegenin altı neredeyse sıfırlanan yüksek frekanslardır; köşegenin üst tarafında sıfırdan farklı katsayılar bulunur. Amaç sıfırları çoğaltıp onları yok etmektir.

> **Ders sonrası not:** Hoca bir yerde "AC'leri insan gözü algılayamıyor, çünkü bunlar yüksek frekans" dedi (01:02:38). Slayta göre algılanmayan, AC katsayılarının yüksek frekanslı olanlarıdır (sağ alttakiler); sol üst bölgedeki sıfırdan farklı AC'ler korunur.

9.4 128 kaydırması

8×8'lik bloğun içinde 0-255 arası renk değerleri vardır. DCT -128 ile $+127$ arasında çalıştığı için bütün değerlerden 128 çıkarılır:

- En küçük değer: $0 - 128 = -128$.
- En büyük değer: $255 - 128 = 127$.

Böylece normalize edilmiş bu matrise T matrisi denir.

9.5 M matrisi ve D matrisi

DCT katsayılarını üretmek için bir M matrisi kullanılır (ekrandan, 01:00:00):

$$M_{i,j} = \begin{cases} \dfrac{1}{\sqrt{N}} & i = 0 \\ \sqrt{\dfrac{2}{N}} \cos\left[\dfrac{(2j+1)i\pi}{2N}\right] & i > 0 \end{cases}$$

i ve j matrisin satır ve sütun indisleridir. N matrisin boyutudur: 8×8'lik bloklarda $N = 8$, 32×32'likte $N = 32$. $i = 0$ ise o gözeye $1/\sqrt{N}$ yazılır, $i > 0$ ise kosinüslü ifade hesaplanır. Slaytta D matrisi (DCT katsayılarından oluşan matris) $D = M^T T M$ diye verilmişti; hoca da "M'nin transpozu ile T matrisini, sonra M ile çarptığımda D matrisini elde ediyorum" diye anlattı.

> **Ders sonrası not:** Hoca indislerin "0,0'dan başlayıp 8,8'e kadar gideceğini" söyledi (01:05:31); 8×8'lik bir matriste indisler 0'dan 7'ye kadardır.

> **Ders sonrası not:** Slayttaki M tanımında i satırı frekansı gösterir. Bu tanımla python'da rastgele bir 8×8 blok üzerinde denendiğinde doğru iki boyutlu DCT'yi $D = MTM^T$ verdi; slayttaki $D = M^T T M$ sırası doğru sonuç vermedi. Ters dönüşüm için slayttaki $M^T R M$ sırası ise bu tanımla doğrudur.

10. Nicemleme, Kodlama ve Ters İşlem

10.1 Nicemleme

Hoca nicemlemeyi (quantization) genel olarak "azaltmak" diye tanımladı. Örneğin renkleri azaltmak için k-means kümeleme algoritması kullanılabilir; aslında yapılan işlem de bir nicemlemedir.

Slayttaki bilgiler (ekrandan, 01:07:40):

- D matrisine önceden hazırlanmış nicemleme (quantization) matrisi uygulanır: $C_{i,j} = \text{round}\left[\dfrac{D_{i,j}}{Q_{i,j}}\right]$.
- DCT dönüşümü sonrasında ortaya çıkan katsayılar genellikle ondalıklıdır, niceleme yöntemiyle tam sayıya dönüştürülür.
- Nicemleme yönteminde her katsayı 1 ile 255 arasında bir tam sayıya bölünür ve daha sonra en yakın tam sayıya yuvarlanır (bu esnada kayıp meydana gelir).
- Katsayıların hangi değere bölüneceği niceleme tabloları (8×8 büyüklüğünde bir nicemleme matrisi) sayesinde belirlenir.
- Parlaklık ve renklilik için farklı tablolar kullanılır.

- Görüntüdeki kayıp miktarını belirlemek için bir kalite çarpanı (quality factor) kullanılır.
- Kullanılan kalite çarpanına göre tablolar değişiklik gösterir.

Slaytta örnek olarak "75 kalite faktörü için parlaklık tablosu" başlıklı 8×8'lik bir matris vardı. Matrisin sol üstündeki değerler küçük (ilk satır 8, 6, 5, 8, 12, ...), sağ alta doğru değerler büyüyordu (son satır 36, 46, 48, 49, 56, 50, 52, 50).

Hoca hazır Q matrislerinin rastgele oranlar için değil, çalışmalarda belirlenmiş standart oranlar için çıkarıldığını söyledi. D matrisi Q matrisine oranlanır; sonuç ondalıklı sayılar olduğu için yuvarlanır ve tam sayılardan oluşan C matrisi elde edilir. Kayıp bu yuvarlamada gerçekleşir.

Büyük değerlere bölünen sağ alt köşedeki katsayılar yuvarlamayla sıfıra iner. Slayttaki bir sonraki maddeye göre C matrisinin sağ alt köşesi 0 ile doludur, çünkü nicemleme işlemi yüksek frekansları yok eder (ekrandan, 01:01:00).

> **Ek bilgi (derste anlatılmadı):** Q matrisinde sağ alttaki değerlerin büyük olması bilinçlidir: Yüksek frekanslı katsayılar büyük sayılara bölündüğü için çoğu sıfıra yuvarlanır, düşük frekanslı katsayılar küçük sayılara bölündüğü için daha hassas korunur. Kalite faktörü düştükçe tablodaki değerler büyür ve daha çok katsayı sıfırlanır.

10.2 Zigzag ve kodlama

Slayta göre C matrisine zigzag coding uygulanır, arkasından değişken uzunluklu kodlama kullanılarak sıkıştırılır (ekrandan, 01:01:00). Hocanın anlatımına göre AC katsayıları çoğunlukla sıfır olduğu için RLE ile sıkıştırılır; DC katsayılarında renk kodları olduğu için Huffman kullanılır ve sıkıştırılmış veri yazılır.

> **Ders sonrası not:** Slayttaki mimaride DC katsayıları için DPCM, AC katsayıları için RLE ve ardından QM kodlaması yer alıyordu (ekrandan, 00:50:00). Hocanın sözlü anlatımındaki Huffman, büyük olasılıkla temel (baseline) JPEG'in entropi kodlamasıdır; QM kodlayıcı aritmetik kodlama seçeneğidir.

10.3 Ters işlem

C'yi bulmak için D, Q'ya bölünüp yuvarlanmıştı. Ters işlemde C matrisi bu kez Q ile çarpılır ve R elde edilir. İleri dönüşümde M'ler ile çarpılan D'nin yerine bu kez R kullanılır, ardından çıkarılan 128 geri eklenir (ekrandan, 01:01:00):

$$R_{ij} = Q_{ij} \cdot C_{ij}$$

$$N = \text{round}(M^T \times R \times M) + 128$$

Böylece DCT'nin tersi elde edilir. Ardından ileri yönde yapılan her işlemin tersi yapılır; istenirse YUV'dan RGB'ye çevrilir ve kaynak resim elde edilir. Kayıplı sıkıştırma yapıldıysa bozulmalar olabilir, ama değiştirilen kısımlar gözün algılamadığı yüksek frekanslar olduğu için JPEG'in kaybını göz pek algılamaz. Çok büyük bozulmalar yapılırsa, örneğin 3000 renk 16 renge düşürülürse, göz bunu elbette algılar; bir odaya bakınca bile 16 farklı rengi ayırt edebiliriz.

11. JPEG 2000

Slayttaki maddeler (ekrandan, 01:12:30):

- Kayıpsız JPEG2000, Joint Photographic Experts Group tarafından geliştirilmiştir.
- Wavelet teknolojisini temel alan en iyi kodlama algoritmasıdır.
- JPEG standardının kısıtlamalarını gidermek ve düşük bit oranlarında yüksek kalitede görüntüler elde etmek amacıyla tasarlanmıştır.
- Wavelet teknolojisinin avantajı sayesinde, DCT'de olduğu gibi sadece kayıplı sıkıştırma değil, kayıpsız sıkıştırma da yapabilmektedir.
- JPEG2000, istemci/sunucu görüntü uygulamaları için ve sınırlı kaynağa sahip olan kablosuz cihazlar için ilave fonksiyonel özelliklere sahiptir.

Hocaya göre JPEG 2000'in JPEG'den tek farkı DCT yerine wavelet dönüşümünü kullanmasıdır. Wavelet dönüşümü sinyal işlemede gösterilen bir konudur ve hazır olarak kullanılır. Daha önce de söz edildiği gibi bu dönüşümler renkleri zaman domeninden frekans domenine çevirir; kosinüs dönüşümü, wavelet dönüşümü ve başka dönüşümler kullanılabilir.

Hoca wavelet dönüşümünü ekranda bir kare çizerek anlattı (ekrandan, 01:12:30). Resim dörde bölünür. Hoca ezbere bilmediğini belirterek bunların büyük olasılıkla low-low, low-high, high-high ve high-low filtreleri olduğunu söyledi. Sonra bu parçalar kendi içinde yeniden dörde bölünür ve seviyesine göre bir takım kayıplar uygulanır. Hocaya göre DCT'den daha iyi sıkıştırma performansı verir. Bu da grup tarafından geliştirildiği için lisanslıdır: Ticari bir ürün piyasaya çıkarılacak ve bu format kullanılacaksa ücret ödenmesi gerekir; aksi halde açık bir format kullanılır.

> **Ders sonrası not:** Hoca JPEG 2000'in DCT'den daha hızlı çalıştığı için kullanıldığını söyledi (01:12:52). JPEG 2000 büyük olasılıkla JPEG'den daha fazla hesaplama gerektirir; slayttaki üstünlükler hız değil, düşük bit oranlarında kalite ve kayıpsız sıkıştırma desteğidir.

12. GIF, TIFF ve PNG

12.1 GIF

Slayttaki maddeler (ekrandan, 01:14:00):

- CompuServe tarafından geliştirilmiştir (1987).
- LZW tabanlı çalışan bir algoritmadır.
- En fazla 8 bit renk derinliğine ($2^8 = 256$ renk) sahip görüntülerde çalışır.
- Renkli fotoğraf görüntülerinin sıkıştırılmasında yetersiz kalır.
- Birkaç rengin çoğunlukta olduğu grafiksel gösterim, basit şekiller gibi görüntülerin kayıpsız olarak sıkıştırılmasında kullanılır.
- 87a ve 89a diye iki versiyonu vardır. 89a, tek bir GIF dosyası içerisinde birden çok GIF formatlı resim yerleştirilmesine ve anime edilmesine olanak tanır.
- Katmanlı görüntü saklama özelliğine sahiptir.
- Kullanım oranı düşüktür (patent koruması altında).
- Donanımdan bağımsız çalışır.

GIF (Graphics Interchange Format) 256'dan fazla renk içeren resimlerde çalışmaz; yüksek kaliteli resimlerde 256'dan fazla renk olduğu için bunlar GIF'e çevrilemez. Animasyonlu GIF'ler 89a sürümüyle mümkündür: Bir resmin içine başka bir resim yerleştirilebilir.

Katmanlı saklamayı hoca şöyle açıkladı: Resim seviyeler halinde saklanır ve en son seviyede en net haliyle görülür. Büyük bir resmin birinci seviyesi açıldığında resim bulanık görünür ama ne olduğu tahmin edilebilir. Aranan resim bu değilse hemen kapatılır; resmin tamamını açmak hem süre hem işlem olarak uzun sürerdi. Bu aslında önce yalnız alçak frekansları göstermek, sonra biraz yüksek frekans katmak ve sonraki aşamalarda daha yüksek frekanslar katarak resmi netleştirmektir.

12.2 TIFF

Slayttaki maddeler (ekrandan, 01:16:30):

- Aldus Corporation tarafından geliştirilmiştir, şu an telif hakkı Adobe Systems şirketine aittir.
- 1988'de yayınlanan 5.0 sürümü ile RGB bitmap verilerini LZW algoritması kullanarak sıkıştırır.
- 1992'de yayınlanan 6.0 sürümü ile JPEG sıkıştırması da eklenmiştir.
- TIFF-LZW'nin de patent sorunlarından dolayı TIFF altında ZIP sıkıştırması geliştirilmiştir.
- TIFF 24 bit renk derinliğini desteklemesi sayesinde fotoğrafların kayıpsız olarak sıkıştırılmasında kullanılır.
- Çok renkli resim dosyalarında zaman yerden daha önemliyse hızından dolayı tercih edilir.
- JPEG'e göre daha az küçülme sağlamasına rağmen hızı ile açığı kapatır.
- TIFF formatının GIF'ten temel farkı; hem 256 sıralı renk görüntülerle, hem de 24 bit RGB (8 bit Red + 8 bit Green + 8 bit Blue) ile çalışabilmesidir.

TIFF (Tagged Image File Format) kayıpsız bir sıkıştırmadır ve lisanslıdır. Hoca tıbbi görüntülerde GIF değil TIFF ile karşılaşılabileceğini söyledi. TIFF'te her palet 8 bittir ve 24 bit derinliği destekler; GIF ise 256 renkle sınırlıdır. 24 bit desteği sayesinde fotoğraflarda kayıpsız sıkıştırma da kullanılabilir.

> **Ders sonrası not:** Hoca "yer daha önemliyse, zaman değil de daha az yer kaplamasını istiyorsam TIFF'i tercih edebilirim" dedi (01:16:34). Slayta göre TIFF, çok renkli resimlerde zaman yerden daha önemliyse hızı yüzünden tercih edilir; JPEG'e göre daha az küçültür.

12.3 PNG

Slayttaki maddeler (ekrandan, 01:18:00):

- GIF ve altyapısındaki LZW algoritmalarının patent anlaşmaları ile koruma altında olmaları, ücretsiz olarak kullanılabilecek bir kayıpsız görüntü sıkıştırma standardı ihtiyacını gündeme getirmiştir. Thomas Boutell, Scott Elliott, Mark Adler, Tom Lane ve birçok başka girişimci tarafından geliştirilmiştir (1996).
- 48 bit gerçek renk, 16 bit gri tonlama desteği ve yüksek oranda sıkıştırma becerisi sayesinde GIF ve TIFF standartlarından daha iyidir.
- Altyapısında ZLIB ve DEFLATE veri sıkıştırma teknikleri yer almaktadır.

PNG (Portable Network Graphics) ücretsiz bir formattır; telif hakkı ödemek istemeyenler bunu kullanır. "Portable" (taşınabilir) olması her türlü donanımda kullanılabilmesini ifade eder.

> **Ders sonrası not:** Hoca DEFLATE'in içinde "LZW ve Huffman" olduğunu söyledi (01:17:39). Dersin başındaki slayta ve hocanın kendi anlatımına göre DEFLATE, LZ77 ile Huffman'ın birleşimidir (ekrandan, 00:07:00).

13. Kapanış: Görüntü Sıkıştırmanın Arka Planı

Hoca, bu formatların hepsinin bilinen yöntemlerin farklı birleşimlerinden oluştuğunu vurguladı. Resmin kalitesine, ne kadarlık sıkıştırma yapılacağına, hızın mı yerin mi önemli olduğuna göre farklı kombinasyonlar ortaya çıkar. Yüksek frekanslı renkler düşürülebilir ya da düşürülmeyebilir; down-sampling yapılarak küçük bloklarda 4 değer yerine 2 değer komşu değerlerin benzerliğine bakılarak kullanılabilir. Bunların farklı türevlerinden oluşmuş bir sürü algoritma vardır.

Görüntü sıkıştırmada anlatılanların ana ve öz olanlar olduğunu, daha birçok şey bulunabileceğini, ama altta yatanların diğer algoritmalarda görülen yöntemler olduğunu söyledi. Sıkıştırmadan önce sinyal işlemede görülen bazı transformları ve bazı nicemleme yöntemlerini bilmek gerekebilir. Bunlar farklı derslerin konularıdır; bilinse de bilinmese de sıkıştırmada hazır olarak alınıp kullanılabilir.

> **Hocanın vurgusu:** Veri çok özelse ve algoritmanın içinde değişiklik yapmak gerekiyorsa, algoritmalar hakkında kısa da olsa bilgi sahibi olmak gerekir; böylece koda girildiğinde nereye ne ekleneceği ya da nereden ne çıkarılacağı rahatlıkla bilinir.

Örnek Sorular

Soru 1 (ekrandan, 00:11:45): 1 bitlik renk derinliğine sahip (yalnız siyah ve beyaz) bir görüntünün bir satırındaki 640 pikselin renkleri sırasıyla 200 beyaz, 240 siyah ve 200 beyaz ise bu satır RLE ile nasıl kodlanır? Çözücü ilk pikselin rengini nasıl bilir?

> **Çözüm (derste verildi):** Yalnız tekrar sayıları yazılır: $(200, 240, 200)$. Toplam $200 + 240 + 200 = 640$ pikseldir. Çözücü ilk pikselin rengini bilmelidir: İlk pikselin rengi kodlamanın başında belirtilir ya da ilk piksel her zaman beyaz kabul edilir; ilk sayı beyaz, ikincisi siyah, üçüncüsü beyaz koşunun uzunluğudur.

Soru 2 (derste sözlü, 00:15:11): İlk piksel her zaman beyaz kabul ediliyorsa, beyazla değil siyahla başlayan bir satır (200 siyah, 240 beyaz, 200 siyah) nasıl kodlanır?

Çözüm (derste verildi): Başa sıfır yazılır: $(0, 200, 240, 200)$, yani "sıfır beyaz, 200 siyah, 240 beyaz, 200 siyah". Gri tonlu bir resimde başlangıç rengi başlıkta belirli bir byte'a yazılır.

Soru 3 (derste sözlü, 00:27:40): LZW nasıl bir algoritmaydı, LZ77 ve LZ78'den farkı neydi?

Çözüm (derste kısmen verildi): Hoca LZW'yi şöyle hatırlattı: Sözlük kullanan bir yöntemdir. Sözlük alfabedeki harflerle başlar; ardından harflerin kombinasyonlarına bakılır, sözlükte olmayan kombinasyon sözlüğe eklenir ve en uzun kombinasyonlar oluşturulana kadar devam edilir. LZ77 ve LZ78 ile farkı bu derste açıkça söylenmedi. Önceki dersin notlarına göre LZ77 önceden kodlanmış veriyi kayan bir pencerede sözlük olarak kullanır ve (offset, uzunluk, sembol) yazar; LZ78 boş başlayan bir sözlük kullanır ve (sıra, sembol) yazar; LZW ise sözlüğü alfabeyle başlatır ve çıkışa yalnız sözlük indeksini yazar.

Soru 4 (ekrandan, 00:55:10): Ekrana yazılan iki pikselin R, G, B değerleri $(45, 55, 42)$ ve $(45, 55, 41)$'dir. Slayttaki formüllerle bu iki pikselin Y ve U değerlerini bulun.

Çözüm (derste verilmedi): Python ile hesaplandı. Birinci piksel: $Y = 0{,}299 \cdot 45 + 0{,}587 \cdot 55 + 0{,}114 \cdot 42 = 13{,}455 + 32{,}285 + 4{,}788 = 50{,}528$; $U = -0{,}169 \cdot 45 - 0{,}331 \cdot 55 + 0{,}500 \cdot 42 = -7{,}605 - 18{,}205 + 21 = -4{,}81$. İkinci piksel: $Y = 50{,}414$, $U = -5{,}31$. İki pikselin parlaklık farkı yalnız $0{,}114$ birimdir; göz bu farkı ayırt edemez. (V için slayttaki katsayı büyük olasılıkla hatalı olduğundan V hesaplanmadı; bkz. Bölüm 8.2.)

Soru 5 (derste sözlü, 01:03:34): Bir bloktaki renk değerleri 0-255 arasındadır ve DCT -128 ile $+127$ arasında çalışır. Değerler bu aralığa nasıl getirilir?

Çözüm (derste verildi): Bütün değerlerden 128 çıkarılır: $0 - 128 = -128$, $255 - 128 = 127$. Elde edilen matrise T matrisi denir. Ters işlemde 128 geri eklenir.

Dersin Toparlaması

Bu derste görüntü sıkıştırma ele alındı. Görüntüde de kayıplı ve kayıpsız sıkıştırma vardır ve kullanılan yöntemler metinde görülen yöntemlerin görüntüye uyarlanmış hâlleridir: RLE siyah-beyaz ve az renkli görüntülerde başarılıdır; faks makineleri metni görüntü olarak görüp Modified Huffman (G31D) ve MMR (G32D) ile kayıpsız sıkıştırır; ZIP/GZIP ve PK programları LZ77, LZW, Huffman, Shannon-Fano ve RLE'yi farklı birleşimlerle kullanır. Kayıpsız JPEG-LS, her pikseli W, NW, N ve NE komşularından iki aşamada tahmin edip yalnız tahmin hatasını kodlar.

Dersin ana bölümü JPEG'di: RGB'den YUV'a geçiş, renklilik bileşenlerinde örneklemeyi azaltma, 8×8 bloklar, 128 kaydırması, DCT, Q matrisiyle nicemleme, zigzag tarama ve RLE/DPCM ile entropi kodlaması. Kayıp nicemlemedeki yuvarlamadan doğar ve gözün algılayamadığı yüksek frekanslara yöneltilir. JPEG 2000 DCT yerine wavelet kullanır; GIF, TIFF ve PNG ise LZW, JPEG ve DEFLATE gibi bilinen yöntemlere dayanan formatlardır. Hoca, görüntü sıkıştırmanın altında daha önce görülen yöntemlerin yattığını, transform ve nicemleme yöntemlerinin hazır kullanılabileceğini, ama özel veride bu algoritmaların iç yapısını bilmenin gerektiğini vurgulayarak konuyu kapattı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.